

Résumé de Git selon l'évolution sur w3Schools

1. Installation de Git :

Sur Ubuntu,

- **La commande :** `sudo apt update && sudo apt-get install git`

Sur Windows,

- Téléchargez la dernière version de git sur le site internet.
- Installer le logiciel téléchargé.
- **Vérification :** `git --version`

2. Configuration : Identité de développeur

- **Identité :** `*git config --global user.name "Genie Logiciel"`
 - `git config --global user.email "genielogiciel.dic"`
- **Vérifiez tout :** `git config --list`. C'est ici que tu vois si tes réglages sont corrects.

3. Git Démarrer et nouveaux fichiers

- **git init :** Cette commande crée le dossier caché `.git`. C'est le cerveau du projet.

Si vous supprimez ce dossier, votre projet redevient un simple dossier sans mémoire.

- **git status :** C'est la commande qu'on tapera le plus. Elle nous dit quels fichiers sont :
 - *Untracked* (nouveaux ou non suivis, Git ne les regarde pas encore).
 - *Modifié* (changés mais pas encore préparés).
 - *Mise en scène* (prêts à être enregistrés).

4. L'Environnement de préparation

Git ne sauvegarde pas tout notre dossier d'un coup, il ne sauvegarde que ce que tu as "ajouté".

- **git add <fichier> :** Ajoute un fichier précis.
- **git add . :** Ajoute tout (fichiers modifiés et nouveaux).
- **git add -u :** Ajoute uniquement les fichiers **déjà suivis** qui ont été modifiés ou supprimés

5. Le Commit : L'enregistrement définitif

Un commit est un "instantané" (instantané) de votre code.

- **git commit -m "Message"**: La méthode classique.
- **git commit -a -m "Message"**: Comme on l'a vu, c'est le combo add + commit. **Attention** : Il ne prend pas les fichiers "Untracked" (nouveaux). Il prend en compte les fichiers déjà préparés.
- **git log** : permet de voir la liste des commits
- **git commit --amend**: Très puissant. Si tu as fait une de frappe dans ton dernier message de faute commit ou si tu as oublié d'ajouter un petit fichier, tu peux "corriger" le dernier commit sans en créer un nouveau.

6. Git Stash : Mettre en pause

Imaginez que le Patron arrive et te demande de corriger un bug urgent sur la branche `main`, mais on es en plein milieu d'un gros changement pas fini sur ta branche actuelle.

- **git stash**: Tes modifications disparaissent et sont stockées dans une pile. Ton dossier devient "propre".
- **git stash pop**: Une fois le bug corrigé, on reviens sur ta branche et tu tapes ça pour faire réapparaître ton travail là où tu l'avais laissé.

7. Git tag: Donner une étiquette

Contrairement aux branches qui évoluent à chaque nouveau commit, un **Tag** est une étiquette fixe attachée à un commit spécifique. C'est un "signet" qui ne bouge plus. Dans les projets, cela servira à marquer la fin d'une étape (ex: "TP1_Terminé"). Notez que que par défaut une étiquette **Tag**

existe que sur votre machine physique en local.

Nous connaison pour le moment **Deux Types de Tags**

- **Tag légé**
 - **Commande : git tag <nomTag>**
- **Tag annoté**
 - **Commande : git tag -a <nomTag> -m "message"**
 - **git tag -a <nomTag> <codeHexaDuCommit> - m "message"** : Pour étiquetteer un commit existant déjà
- **git tag** : Pour afficher la liste des étiquettes déjà définies
- **git show <nomTag>** : Pour afficher les informations sur une étiquettes données

Pousser une éttiquette en ligne sur le dépôt distant

- **git push origin <nomTag>**
- **git push origin --tags** : Pour pousser tous les tags existants.

7. Historique et Aide

- **git log** : Pour voir l'historique de commits effectués
- **git log --oneline --graph**: Pour voir l'historique comme une carte routière.
- **git log -p**: Affiche carrément les lignes de code qui ont changé dans chaque commit.
- **git help <commande>**: Ouvre la documentation complète (dans ton navigateur ou terminal).

8. Branches et Fusion (Merge)

- **git branch <nom>**: Crée la branche.
- **git checkout -b <nom>**: Crée la branche **ET** se déplace dessus immédiatement (un gain de temps énorme).
- **git switch <nomBranch>** : Pour switcher de la branche de travail actuel vers la branche <nomBranch>
- **git merge <nomBranch>**: Pour fusionner.